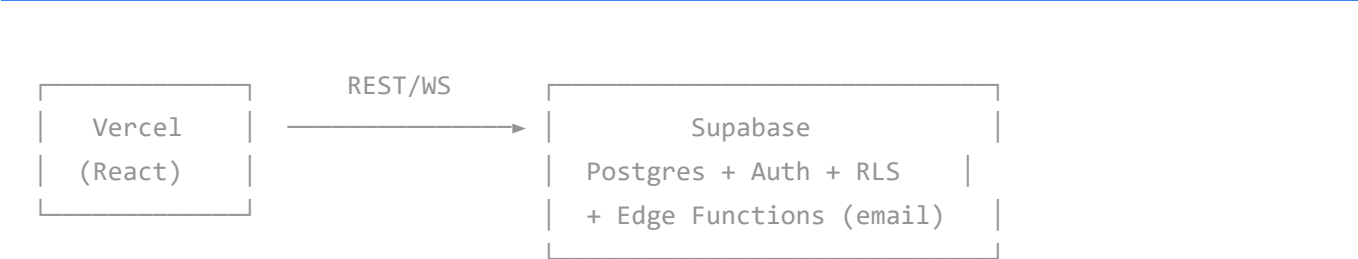


Cadence — Technical Documentation

AtomQuest Hackathon 1.0

Architecture



Database Schema

Table	Purpose	Key Fields
users	Employee/Manager/Admin profiles	id, name, email, role, department, manager_id
cycles	Performance cycles	id, name, phase, status, open_date, close_date
goals	Employee goals	id, employee_id, cycle_id, title, description, thrust_area, uom_type, target, weightage, status, locked_at
checkins	Quarterly progress	id, goal_id, quarter, actual_achievement, score, notes
checkin_comments	Manager feedback	id, checkin_id, manager_id, comment
audit_logs	Action history	id, action, target_table, target_id, changed_by, details
escalation_rules	Nudge configuration	id, rule_type, target_role, threshold_days, is_active
escalation_logs	Sent notifications	id, rule_id, target_id, triggered_at, status

RLS Policies (Security)

Row-Level Security enforces role access at the database layer:

- `goals` — employees see their own; managers see direct reports'; admin sees all
- `checkins` — same as goals
- `users` — employees see self + manager; managers see team; admin sees all
- `audit_logs` — admin read-only

Triggers

Trigger	Table	Purpose
validate_weightage_sum	goals	Blocks INSERT/UPDATE if total weightage per employee per cycle exceeds 100%
audit_log_insert	goals , cycles	Auto-records every create, update, delete to audit_logs
checkin_score_compute	checkins	Auto-calculates score based on UoM formula on INSERT/UPDATE

UoM Scoring Formulas

Min = (target / actual) * 100
capped = min(score, 100)

Max = (actual / target) * 100
capped = min(score, 100)

Zero = 100 - (actual / target * 100)
capped = max(score, 0)

Timeline = (completed_milestones / total_milestones) * 100

Milestone = same as Timeline (binary gates)

Final weighted score per goal = (raw_score / 100) * weightage

API Layer

No custom backend. All data access via Supabase JS client:

```
supabase.from("goals").select("*").eq("employee_id", uid)
supabase.from("goals").update({ status: "submitted" }).eq("id", id)
supabase.auth.signInWithPassword({ email, password })
```

State Management

Concern	Tool	Pattern
Server state	TanStack Query	<code>useQuery</code> with 5s <code>staleTime</code> , auto-cache invalidation on mutations
Client state	Zustand	<code>authStore</code> : session, profile, <code>demoRoleOverride</code>
URL state	React Router v6	Protected routes, role-based redirects

Caching Strategy

- **staleTime:** 5000ms — data fresh for 5s after fetch, prevents re-render loops
- **gcTime:** 5 minutes — inactive queries cached in background
- **Mutations:** Call `invalidateQueries()` on success to force refresh

Frontend File Structure

```
src/  
  pages/           # Public pages (Landing, Login)  
  features/        # Domain modules (goals, checkins, team, admin, analytics, escalations)  
  components/      # Shared UI (AppShell, buttons, cards)  
  stores/          # Zustand auth store with localStorage persistence  
  hooks/           # Reusable TanStack Query hooks  
  lib/             # Utils, scoring formulas, CSV export, Supabase client  
  types/           # TypeScript schema mirror
```